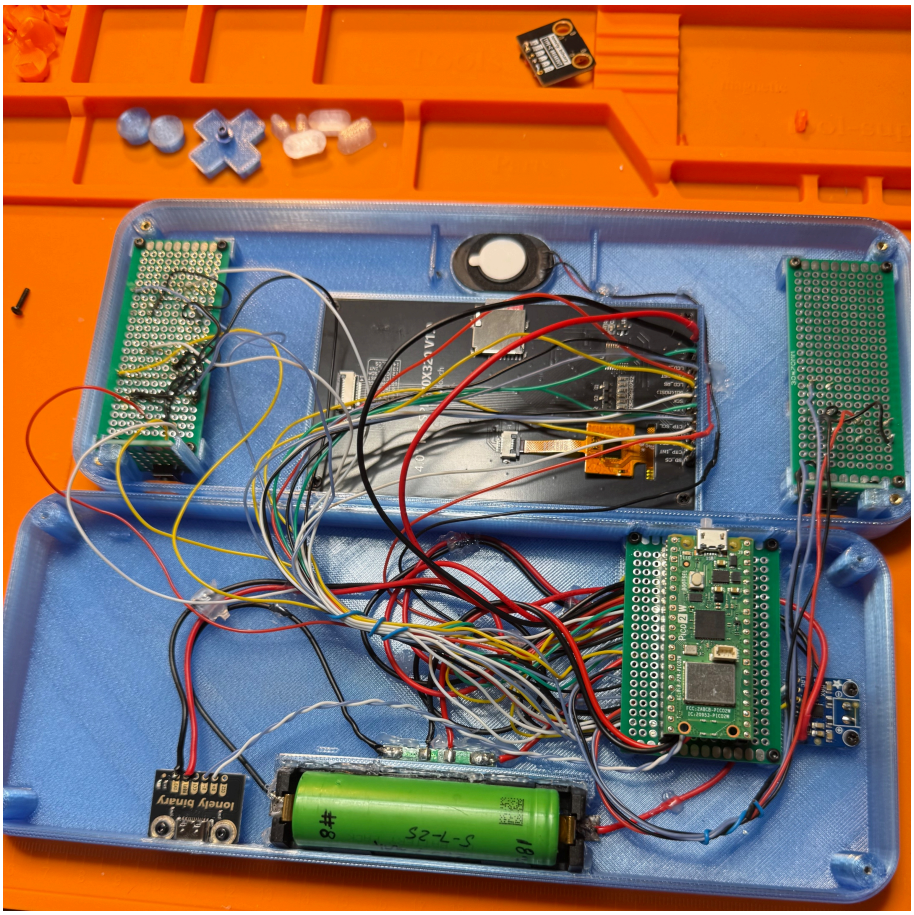
Pico 2 W handheld for Game Boy and Game Boy Color games: build guide

A handheld that plays Game Boy and Game Boy Color games from a microSD card. It has a 4" touch screen, 10 buttons, a speaker with volume control, and an 18650 battery that charges over USB-C. It's built around a Raspberry Pi Pico 2 W in a 3D-printed shell shaped like a Game Boy Advance.

No games are included. Use game files you own.



What it does

- **Games:** Game Boy (.gb) and Game Boy Color (.gbc) files up to 2 MB each, from the microSD card. Bigger games don't fit the Pico 2 W's flash and are left off the list.
- **Menu:** tabs for special games, Recent, Favorites, Genres, A-Z and Settings, with a cover picture or a name tile for every game. A small built-in game, Night Survivors, is on the first tab.
- **Screen:** 4.0", 480 x 320, ST7796S. The game picture is drawn without tearing.
- **Buttons:**
  - D-pad (4 separate pieces), A, B, Start, Select, L, R.
  - Hold **Start + Select** for about 1 second to go back to the menu from a game.
- **Touch:** in the menus (tap tabs, games, settings; swipe to scroll). It needs a finger: the glass is capacitive.
- **Brightness:** 10 levels. Start + Select + A (brighter) / B (dimmer), anywhere, or the Settings tab.
- **Volume:** 8 levels, 0 = off, starting quiet. Hold Start and press R (louder) / L (quieter), anywhere, or Settings.

- **Settings tab:** brightness, volume, PIN lock, parent code, theme (Classic, Galaxy, Jungle, Lava). Everything is saved on the card.
- **PIN lock:** a 4-digit PIN asked at every start; 5 wrong tries in a row give a 30 s wait. A parent code also opens it. Deleting `/saves/lock.dat` on the card (on a computer) clears both.
- **Screen off on USB:** plugged in and untouched for 2 minutes, the screen goes dark (the game keeps running); any button or touch brings it back without doing anything else.
- **Battery:** one 18650 cell, charged from USB-C while you play (Pimoroni LiPo SHIM). The menu shows the battery level, estimated from the cell's voltage.

Size

234.2 x 94 x 29.5 mm.

Parts

Prices are what this build paid in October 2026. Check current prices.

Qty	Part	Where	Notes
1	Raspberry Pi Pico 2 W, headers soldered on	any Pico reseller	plugs into a socket board
1	Hosyond 4.0" 480x320 SPI display, ST7796S, capacitive touch (FT6336U), board "V1.0", with a microSD slot	Amazon (Hosyond's 4" capacitive listing is B0CRGQN58D: check it says ST7796S, capacitive, with the SD slot)	the slot on the screen board holds the games card
1	Pimoroni LiPo SHIM for Pico (PIM557)	Adafruit 5612, $8.95	power button, charging, battery protection
1	MAX98357A I2S amplifier breakout	Adafruit 3006	with its screw terminal
1	Mini oval speaker, 8 ohm 1 W, 30 x 20 x 5 mm	Adafruit 3923, $1.95	cut off its tiny plug
6	Soft tactile buttons, 7.8 x 7.8 mm, through-hole	Adafruit 3101 (10 pack $1.95)	D-pad x4, A, B
4	Tactile switches, 6 x 6 mm, through-hole	Adafruit 367 (20 pack $2.50)	L, R, Start, Select
1	USB-C breakout with 5.1k CC resistors (Lonely Binary)	Amazon B0GTQH9XKP (12 pack $12.99)	the charging and data port
1	18650 cell, unprotected, high capacity (e.g. LG MJ1 or Samsung 35E)	a real battery shop	avoid marketplace listings: anything claiming 5,000+ mAh is fake
1	Single 18650 holder, surface-mount style (a flat solder tab out of each end)		the bay is modelled to one 86.54 x 21.03 x 15.19 mm with its tabs
1	1-cell protection strip (DW01-A + 8205A), pads B+ B- P+ P-		30.16 x 4.05 mm
1	JST-PH 2-pin cable		strip to the SHIM's battery plug
2	Perfboard 30 x 70 mm, 1.52 mm thick		the two button boards, used whole

Qty	Part	Where	Notes
1	Perfboard 40 x 60 mm		the Pico's socket board; offcuts make the L and R pieces (6 x 6 holes each)
2	Female header strip, 1 x 20		the Pico's socket
26	Brass heat-set inserts, M2 x 4 mm long x 3.2 mm across		18 in the front half, 8 in the back half
20	M2 x 6 screws		button boards 8, screen 4, L/R 2, socket board 4, amp 2
6	M2 x 8 screws		joining the halves 4, USB-C board 2
	Wire: 30 AWG for signals, 26 or 24 AWG for power and ground, 1 twisted pair for USB D+/D-		
1	microSD card		FAT32 (up to 32 GB) or exFAT
	PETG filament, for every part		PLA softens in a hot car

Tools: soldering iron (also sets the inserts, at about 230 °C), multimeter, calipers, flush cutters, hot glue, small Phillips screwdriver.

Printing

Every file already lies the way it prints.

File	What	How it prints
`front_half.stl`	front half	face down, no brim
`back_half.stl`	back half	back face down, Outer brim 5 mm
`power_button_plunger.stl`	power button plunger (2, one spare)	as loaded
`usb_c_sleeves.stl`	4 sleeves for the USB-C board's holes (2 spare)	as loaded
`button_L.stl`, `button_R.stl`	L and R	flat side down
`buttons_ABSS.stl` + `buttons_ABSS_letters.stl`	A, B, Start (+), Select (-) with the letters in a second colour	load both together, answer **Yes** to "single object with multiple parts", give the letters the second colour
`button_A.stl`, `button_B.stl`, `button_start.stl`, `button_select.stl`	the same 4 without letters, for one colour	flat side down
`dpad_4_pieces.stl`	the D-pad, as 4 separate pieces	flat side down

Settings used (Bambu Studio, X1C, PETG):

- **Shell:** 0.2 mm layers, 4 walls, 5 top and 5 bottom layers, 30% gyroid infill, no supports.
- **Buttons:** 0.08 mm layers (smoother sides, so they don't catch), 4 walls, 12 top and 12 bottom layers, 30% gyroid, no supports, Outer brim 5 mm. Trim any brim edge off: that edge sits in the hole's countersink.

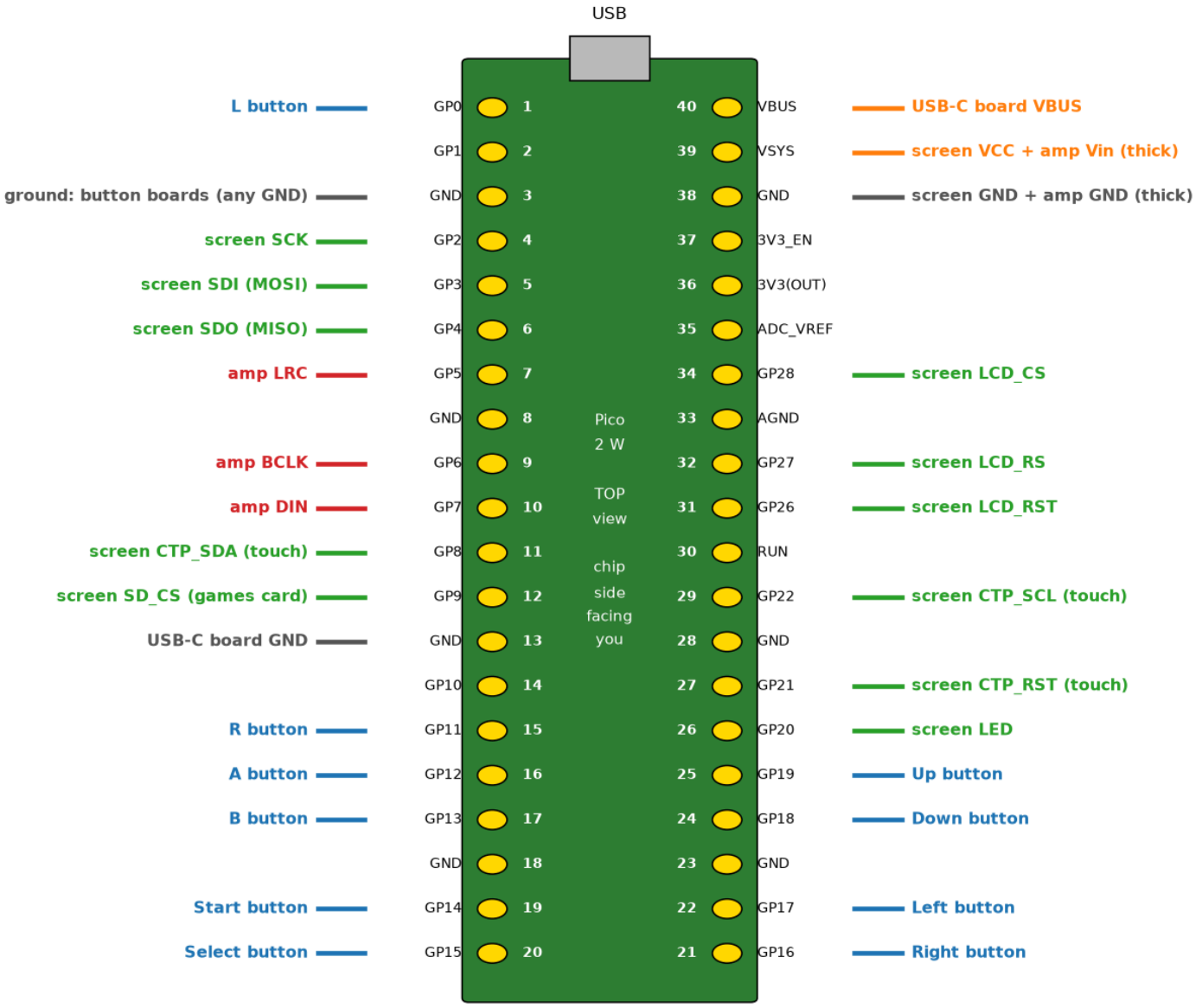
Why the D-pad is 4 pieces: a one-piece D-pad rocking on these switches kept pressing a second direction (Up came on with Right or Left). Split into 4, each piece sits over its own switch and nothing else, and diagonals still work by pressing two pieces. Each piece's top dips 0.5 mm towards the middle, its inner point is cut off, and every top edge is rounded 1.3 mm, so the thumb rests in the middle and rolls between pieces without catching an edge.

Wiring

Every wire goes to the Pico by its **pin number**, the same numbers as Raspberry Pi's Pico 2 W pinout: pin 1 is top left with the USB end up and the chip side facing you, and pins 1-20 run down the left side.

> **Check every wire against the numbers printed on your own boards**, not against a drawing alone.

Handheld wiring: what goes on each Pico pin (the same pins the firmware uses)



Buttons: signal leg to the pin shown, the DIAGONAL leg to ground. USB-C D- to test pad TP2, D+ to TP3.
Pin numbers as in Raspberry Pi's Pico 2 W pinout; check each against the numbers printed on your own boards.

Buttons:

- Each button is a switch between its pin and ground.
- Use one corner leg for the signal and the diagonally opposite leg for ground: those two are always across the switch.
- Each board's ground is one wire to any Pico GND pin: 3, 8, 13, 18, 23, 28 or 38. Pin 33 is the analog ground; leave it out.

Button	Pico pin	GPIO
L	1	GP0
R	15	GP11
A	16	GP12
B	17	GP13
Start	19	GP14
Select	20	GP15
Right	21	GP16
Left	22	GP17
Down	24	GP18
Up	25	GP19

Screen (labels as printed on the screen board):

Screen	Pico pin	GPIO / power
VCC (thick wire)	39	VSYS
GND (thick wire)	38	GND
LCD_CS	34	GP28
LCD_RST	31	GP26
LCD_RS	32	GP27
SDI (MOSI)	5	GP3
SCK	4	GP2
LED	26	GP20 (brightness)
SDO (MISO)	6	GP4
CTP_SCL	29	GP22
CTP_RST	27	GP21
CTP_SDA	11	GP8
CTP_INT	not connected	
SD_CS	12	GP9 (the games card)

Amplifier:

Amp	Pico pin
Vin (thick)	39 (VSYS)
GND (thick)	38

Amp	Pico pin
DIN	10 (GP7)
BCLK	9 (GP6)
LRC	7 (GP5)

The speaker goes to the amp's screw terminal: + to +, - to -.

USB-C board:

- VBUS to **pin 40** (VBUS). GND to **pin 13**.
- D- to the Pico's test pad **TP2**, D+ to **TP3** (the USB data pads in Raspberry Pi's Pico 2 W datasheet). Run them as one twisted pair, kept twisted up to the pads.
- CC and SBU stay unconnected.
- Never plug in the USB-C port and the Pico's own micro-USB at the same time: they share the same 5 V and data wires.

Power:

- The LiPo SHIM solders onto the Pico's pins at the USB end, parts facing out, its button towards the USB end.
- Battery holder + to the strip's **B+**, holder - to **B-**. Strip **P+ / P-** to the JST-PH cable for the SHIM.
- **Check the plug's polarity against the + and - printed on the SHIM before plugging it in.**

Assembly

1. **Inserts:** set all 26 before anything goes in. Iron at about 230 °C; let each sink straight down until flush with the top of its post. Let them cool.
2. **Screen:** into the front half from behind, 4 M2 x 6 through its corner holes.
3. **Button boards:**
    - Solder the switches flat against the boards.
    - The D-pad's 4 switches go 4 holes (10.16 mm) out from the D-pad's centre.
    - Put the buttons in their holes from behind with the front half face down, then lay each board on its 4 posts and screw it down (M2 x 6).
4. **L and R:** each switch on its small perfboard piece, slid along its grooves from behind, one M2 x 6 from underneath.
5. **Speaker:** into its ring behind the grille.
6. **Battery bay:** holder and strip into the back half's bay. A dot of hot glue on the holder's side if it lifts out with the cell. Never glue under it.
7. **USB-C board:** onto its 2 posts with the printed sleeves in its holes, M2 x 8.
8. **Socket board:**
    - Lower it onto its 4 posts 0.5 mm short of the top wall, slide it 1.6 mm up into its groove, then 4 M2 x 6.
    - Drop the power plunger into its slot in the top wall, then plug in the Pico with its SHIM.
9. **Amp:** onto its 2 posts, M2 x 6.
10. **Firmware and first power-up** (next section), with the shell still open: check every button and the sound.
11. **Close:** 4 M2 x 8 from the back into the front half's corner inserts.

Firmware

The ready file is `firmware/pico_games_handheld.uf2` on the project's GitHub page: https://github.com/papershoes22/pico2w-handheld (MakerWorld doesn't take firmware files).

First time (shell open): hold the Pico's BOOTSEL button while plugging its micro-USB into a computer. A drive called `RP2350` appears: copy the .uf2 onto it. The Pico restarts into the menu.

Later, with the shell closed: with the handheld on USB-C, setting its serial port to 1200 baud restarts it into the `RP2350` drive (on Linux: `stty -F /dev/ttyACM0 1200`). Then copy the .uf2 as above.

The games card

1. Format the microSD card FAT32 (up to 32 GB) or exFAT.
2. Copy your .gb / .gbc files into the card's top folder, or one folder down (for example `gb/` and `gbc/`).
3. Make the menu files (below). Run it again whenever you add games.
4. Saves go in `/saves` on the card by themselves.

Making the menu files: it needs Python 3 with Pillow and numpy (`pip install pillow numpy`). Put `make_menu_card.py` and `font8x8_basic.h` (both in the GitHub page's `tools` folder) in one folder, and with the card in the computer run:

```
python3 make_menu_card.py <the card's folder>
```

Every game gets a name tile. **For a cover picture:** put a picture named like the game file into a folder called `covers` on the card (`covers/Tetris DX.png` for `gbc/Tetris DX.gbc`; .png or .jpg), then run the tool again. Tested on Linux only.

Without the menu files the handheld still plays games from a plain list, but without the tabs, covers or the Settings tab.

Troubleshooting

- **Touch does nothing:** use a finger. The touch glass is capacitive.
- **No sound:** check the volume (hold Start, press R).
- **A game is missing from the list:** it's over 2 MB, or not in the card's top folder or one folder down.
- **Screen dark while plugged in:** that's the screen-off after 2 minutes; press any button.

What has and hasn't been tested

- Built and used as one handheld: all buttons, the screen, touch, sound, the card, saves, PIN lock, themes, brightness and volume were checked on it.
- **Screen-off on USB:** checked on the first build. The shared firmware has no Bluetooth, so it tells USB from battery by the voltage alone; that way hasn't been checked yet.
- **Battery:** play time hasn't been measured yet, and the battery level is estimated from the voltage.
- **Known issue:** once, a game started from the menu wasn't kept as the last game played (the start-up screen offered an older one). The cause hasn't been found; the firmware's USB log now says each time it saves or reads it.
- The menu tool's cover pictures and name tiles were checked on a computer copy of the menu.

Credits and licences

The firmware is built on:

- **Peanut-GB** (Mahyar Koshkouei, MIT), the Game Boy emulator.
- **Pico-GB** (YouMakeTech, MIT), the Pico port it started from.
- **I2S sound driver and colour tables** (Vincent Mistler, MIT).
- **minigb_apu** (Alex Baines, Mahyar Koshkouei, MIT), the sound chip.
- **FatFs** (ChaN), which reads the card.
- **Raspberry Pi Pico SDK** (BSD-3-Clause) and **TinyUSB** (MIT).
- **font8x8** (public domain) and **Hedley** (CC0).
- **Press Start 2P** font (SIL Open Font License) for the letters on the buttons.

Every licence's full text is in `LICENSES.txt`, in the zip and on the GitHub page.